Bonus B - Sounds

Add the click, the ding, and the music. It changes everything.



```
collectSound.play();
```

This is the loudest page in the book.

```
    collectSound.play();   // one-shot sound effect on collection
```

Your game is silent. This chapter gives it a ding when you collect a coin, a sound when the game ends, and music underneath the whole thing.

You can take it any time after Chapter 6, once your game has things to collect and a way to end. It pairs well with Bonus A, or works on its own. It takes one meeting.

This chapter does it in three moves.

Part	The idea
One	**A second library.** Install the Sound library, and put your sounds in `data`.
Two	**Declare, load, play.** The same three places as everything else.
Three	**Where the sound goes.** One line of code, and the real thinking is where to put it.

Part One - A second library

B.1 Installing the Sound library

SpriteGame is a library somebody else built, and you've used it since Chapter 1. Sound comes in a second one, from the people who make Processing. You install it once.

Build it. Choose **Sketch → Import Library → Add Library**. Search for "Sound", and pick the one by **The Processing Foundation**. Install it. Then **quit Processing and start it again**.

This is the step most likely to eat your time, so do it first.

Build it. At the top of your sketch, under the sprites import, add the second import:

```
import coxprogramming.processing.sprites.*;
import processing.sound.*;
```

B.2 Your sounds

Your turn. Find two sound files: one short effect, like a ding, a pop, a coin, or a beep, and, if you want one, a music loop about thirty seconds long. Free sounds are at opengameart.org, freesound.org, and zapsplat.com. Start with opengameart.org, which lets you download without an account. freesound.org needs a free account first. If a site won't let you log in, don't fight it. Switch sites, or ask your teacher.

Build it. Sketch → Show Sketch Folder, and drag both into the `data` folder, exactly as in Bonus A. Do it as soon as each one downloads. A sound that's only saved on a website can vanish, and then it's gone from your game too. MP3, WAV, OGG, and AIFF all work. **WAV is the most reliable**, especially for short effects.

Part Two - Declare, load, play

Step 1 - A sound for a coin

A sound goes in the same three places as everything else you've made. Declare it at the top, make it in `setup()`, and use it when you need it.

Build it. At the top, with your other variables:

```
SoundFile collectSound;
```

In `setup()`, load it, with **your** file's name:

```
  collectSound = new SoundFile(this, "collect.wav");
```

In `playGame()`, find the coin's `if`, and add one line after the score:

```
  if (player.collidesWith(coin)) {
    score = score + 1;
    coin.moveTo(random(width), random(height));
    collectSound.play();   // one-shot sound effect on collection
  }
```

Aside: `SoundFile`, and `play()`

A **`SoundFile`** is a variable for a sound, the way a `PImage` is for a picture. `new SoundFile(this, "collect.wav")` reads the file out of `data`. `collectSound.play()` plays it once, from the beginning. Call `play()` again while it's still going, and it starts over.

Run it, and collect a coin.

Check. A ding, every time you touch the coin.

Watch out. Get the file's name wrong, `"colect.wav"`, and Processing prints this under your code as the sketch starts:

```
Sound library error: unable to find file colect.wav
```

The game still runs, right up until the first coin. Then the sketch stops with `NullPointerException`, on the `play()` line. So when a `play()` line stops your sketch, check the folder and the file's name first, every time.

Step 2 - A sound for the end

Build it. Declare and load a second sound the same way, `gameOverSound`. Play it right where you set `gameOver = true;`:

```
  if (player.collidesWith(hazard)) {
    gameOver = true;
    gameOverSound.play();  // play the end sound
  }
```

Check. Touch the hazard, and the game ends with a sound.

Step 3 - Music

Build it. Declare and load a third sound, `bgMusic`, with your music file. Then, at the end of `setup()`, start it:

```
  bgMusic.amp(0.4);   // lower music volume so effects can be heard clearly
  bgMusic.loop();     // start background music - runs until stopped
```

And stop it when the game ends, in the hazard's `if`, before the end sound:

```
    bgMusic.stop();        // stop the music
```

Aside: `loop()`, `stop()`, and `amp()`

`loop()` plays a sound over and over until you call `stop()`. That's what music wants. `amp()` sets how loud: `0.0` is silent, and `1.0` is full volume. Music at full volume drowns out your effects, so turn it down.

Check. Music from the moment the game starts. It stops when the game ends, and the end sound plays.

Play it, and change the `0.4` until the balance feels right.

Part Three - Where the sound goes

B.3 The real skill

The code for playing a sound is one line. Deciding **where** that line goes is the actual thinking. A sound belongs at the exact moment its event happens.

The event	Where `play()` goes
collected something	inside its `if (player.collidesWith(...))`, next to the score
the game ended	right where you set `gameOver = true;`
music, all the time	once, in `setup()`, with `loop()`

That's Chapter 4's idea again. A sound effect is an event: it happens once, when something happens. Music is more like continuous input: it's going the whole time.

> **Watch out.** Put `collectSound.play();` in `draw()`, outside any `if`, and it starts over sixty times a second. It sounds like a machine gun.

> **Watch out.** Use `loop()` for a coin's ding, and it dings forever. `loop()` is for music.

B.4 Make it yours

Your turn. Choose sounds that fit your world. A cartoon boing in a space game is a choice. Make sure it's one you meant.

Your turn. Optional: any of these.

- A sound for a click, or for a key. It goes in `mousePressed()` or `keyPressed()`.
- A sound at a score milestone: `if (score == 10) { levelUpSound.play(); }`, inside the coin's `if`, right after the point.
- Fix the machine gun. When collisions come fast, `play()` restarts over and over, and the sound stutters. Guard it, so it only starts when it isn't already playing:

```
if (!collectSound.isPlaying()) collectSound.play();
```

Then decide which version you like. Sometimes the stutter is the right choice.

> **Watch out.** When you hand in your sketch, zip the whole folder, `data` and all, or nobody hears a thing. Short music loops keep the folder small. A four-minute song makes it huge.

Chapter summary

You built	It's an example of
`import processing.sound.*;`	a second library
`SoundFile collectSound;` and `new SoundFile(this, "collect.wav")`	declare, load
`collectSound.play()` in the coin's `if`	a sound at the moment of its event
`bgMusic.loop()` in `setup()`, and `bgMusic.stop()`	music, all the time, until the end
`bgMusic.amp(0.4)`	balancing the volume

New syntax in this chapter

Syntax	Where you met it
`import processing.sound.*;`	B.1
`SoundFile`, `new SoundFile(this, "...")`	Step 1
`play()`	Step 1
`loop()`, `stop()`, `amp()`	Step 3
`isPlaying()`	B.4

Vocabulary

`loop()` -- plays a sound over and over, restarting when it reaches the end, until `stop()`. (Step 3)

`play()` -- plays a sound once, from the beginning. (Step 1)

`SoundFile` -- a variable for a sound. (Step 1)

`stop()` -- stops a sound. (Step 3)

Check your understanding

1. What two things does a sketch need before it can play a sound?
2. Where does the coin's sound go, and why there?
3. Why does music use `loop()`, and a coin use `play()`?
4. Your game runs, and stops with `NullPointerException` the first time you collect a coin. What's wrong?
5. What does `amp(0.4)` do?